

SYSTEM ENGINEERING COOKBOOK

Cookbook Xây dựng và Vận hành Hệ thống Web Multi-user

Auth, PostgreSQL, kiến trúc, deploy và vận hành — case study
AI Learn JLPT

Case study: AI Learn JLPT

Phiên bản 2026

Mục lục

Lời mở đầu	1
Cách đọc	1
Luật an toàn cho lab	1
1 Mental model của một hệ thống web	3
1.1 Mục tiêu	3
1.2 Bày khái niệm đủ để bắt đầu	3
1.3 Source, build artifact và runtime	3
1.4 Theo dấu một request thật	4
1.5 Liveness, readiness và shutdown	4
1.6 Lab: truy dấu endpoint	5
1.7 Failure modes	5
1.8 Tự kiểm tra	5
2 Thiết kế hệ thống nhiều người dùng	6
2.1 Mục tiêu	6
2.2 Bốn câu hỏi khác nhau	6
2.3 Phân loại dữ liệu trước khi viết API	6
2.4 IDOR và horizontal privilege escalation	7
2.5 Role hay permission?	7
2.6 Lab: ma trận hai user	7
2.7 Failure modes	8
2.8 Tự kiểm tra	8
3 Xây dựng session authentication	9
3.1 Mục tiêu	9
3.2 Authentication flow	9
3.3 Password hash không phải encryption	9
3.4 Session và cookie	9
3.5 Request context và access	10
3.6 Session hay JWT?	10
3.7 Lab: auth bằng <code>curl</code>	10
3.8 Failure modes	11
3.9 Tự kiểm tra	11
4 Làm auth đủ an toàn cho production	13
4.1 Mục tiêu	13
4.2 Threat model tối thiểu	13
4.3 Ưu tiên 1: giới hạn thử credential	13
4.4 Ưu tiên 2: session lifecycle	13
4.5 CSRF và SameSite	14
4.6 Email verification và reset password	14
4.7 Audit, MFA và OAuth	14
4.8 Recipe triển khai theo lát dọc	14
4.9 Lab: review auth hiện tại	15
4.10 Failure modes	15

4.11	Tự kiểm tra	15
5	Cài đặt và kết nối PostgreSQL	16
5.1	Mục tiêu	16
5.2	Mental model	16
5.3	Cài cho development	16
5.4	Tạo role và database	17
5.5	Làm việc với <code>psql</code>	17
5.6	Lab: database cô lập	18
5.7	Chẩn đoán theo lớp	18
5.8	Tự kiểm tra	18
6	SQL thực chiến trên schema dự án	20
6.1	Mục tiêu	20
6.2	Data type là một phần của thiết kế	20
6.3	CRUD có scope	20
6.4	JOIN và quan hệ	20
6.5	Aggregate, CTE và pagination	21
6.6	Transaction	21
6.7	Index và query plan	21
6.8	JSONB: dùng có chủ đích	22
6.9	Lab: query an toàn	22
6.10	Failure modes	23
6.11	Tự kiểm tra	23
7	PostgreSQL bên trong ứng dụng	24
7.1	Mục tiêu	24
7.2	Connection pool	24
7.3	Query và transaction	24
7.4	Migration như release contract	25
7.5	Lock, deadlock và timeout	25
7.6	Backup và restore	25
7.7	Lab: recovery drill	26
7.8	Failure modes	26
7.9	Tự kiểm tra	26
8	Ghép các thành phần thành hệ thống	28
8.1	Mục tiêu	28
8.2	Dependency map	28
8.3	Request flow và failure boundary	28
8.4	Config	29
8.5	Error và log	29
8.6	Build và static assets	29
8.7	Security và resilience là thành phần	29
8.8	Lab: thiết kế endpoint trước khi code	30
8.9	Failure modes	30
8.10	Tự kiểm tra	30
9	Chọn framework, OS và plugin	31
9.1	Mục tiêu	31
9.2	Framework HTTP	31

9.3	Database access	31
9.4	Process và packaging	31
9.5	Reverse proxy và OS	32
9.6	Khi nào thêm thành phần	32
9.7	Dependency gate	32
9.8	Bộ cài tối thiểu	33
9.9	Lab: architecture decision record	33
9.10	Tự kiểm tra	33
10	Deploy và CI/CD	34
10.1	Mục tiêu	34
10.2	Topology VPS đơn	34
10.3	Artifact và release directory	34
10.4	Thứ tự release	35
10.5	Secret và least privilege	35
10.6	systemd và Nginx	35
10.7	Rollback nào?	35
10.8	Lab: đọc release như state machine	36
10.9	Failure modes	36
10.10	Tự kiểm tra	36
11	Vận hành và xử lý sự cố	37
11.1	Mục tiêu	37
11.2	Bốn tín hiệu đầu tiên	37
11.3	Evidence trước action	37
11.4	Playbook: app crash	37
11.5	Playbook: PostgreSQL không ready	37
11.6	Playbook: disk đầy	38
11.7	Playbook: migration/release lỗi	38
11.8	TLS và certificate	38
11.9	Backup operations	38
11.10	Khi nào scale	38
11.11	Lab: game day không phá dữ liệu	39
11.12	Tự kiểm tra	39
12	Command cookbook	40
12.1	Git, dependency và build	40
12.2	PostgreSQL status và khám phá	40
12.3	Database lifecycle	41
12.4	Backup và restore	41
12.5	HTTP và health	42
12.6	systemd và journal	42
12.7	Port, process và resource	42
12.8	Nginx, DNS và TLS	42
12.9	SSH và deploy	43
12.10	Cách dùng cookbook khi incident	43
13	Capstone và checklist	44
13.1	Mục tiêu	44

13.2	Capstone: từ login tới recovery	44
13.2.1	1. Chuẩn bị	44
13.2.2	2. Identity và ownership	44
13.2.3	3. Artifact	44
13.2.4	4. Backup/recovery	45
13.2.5	5. Cleanup	45
13.3	Maturity matrix	45
13.4	Checklist kiến thức	46
13.5	Security review	46
13.6	Database và go-live review	46
13.7	Câu hỏi tình huống	46
13.8	Những thứ chưa cần vội dùng	47

Lời mở đầu

Quyển sách này trả lời một câu hỏi thực tế: từ một codebase đang chạy trên máy cá nhân, làm thế nào để hiểu, xây dựng và vận hành nó như một hệ thống web nhiều người dùng?

Đây không phải tài liệu “copy lệnh rồi hy vọng”. Mỗi quyết định đều đi theo bốn bước: vấn đề là gì, thành phần nào chịu trách nhiệm, cách kiểm chứng ra sao, và khi hỏng thì quan sát ở đâu. Dự án AI Learn JLPT được dùng làm case study vì nó có đủ một vòng đời nhỏ nhưng thực: browser client, native Node.js HTTP server, session authentication, PostgreSQL, migration, build artifact, Nginx, systemd và GitHub Actions.

Cách đọc

Nếu muốn hiểu toàn hệ thống, đọc tuần tự. Nếu đang giải quyết một việc cụ thể:

Nhu cầu	Chương nên đọc
Bổ sung auth hoặc sửa lỗi phân quyền	2, 3, 4
Học PostgreSQL và SQL	5, 6, 7
Hiểu kiến trúc codebase	1, 8
Chọn framework hoặc giảm dependency	9
Đưa hệ thống lên VPS	10, 11, 12
Tự đánh giá kiến thức	13

Các nhãn trong sách có ý nghĩa cố định:

Hệ thống hiện có

Nội dung trong hộp này đã được đối chiếu với source hiện tại. Đường dẫn code là source of truth; sách giải thích ý nghĩa chứ không thay thế code.

Khuyến nghị

Đây là thiết kế nên bổ sung khi yêu cầu production đòi hỏi. Nó không được mô tả như một tính năng đã tồn tại.

Cảnh báo

Lệnh có thể làm thay đổi dữ liệu hoặc availability. Đọc preflight và xác nhận đúng database, host, user trước khi chạy.

Luật an toàn cho lab

1. Lab database chỉ dùng database có tên chứa `lab` hoặc `test`.

2. Không dùng `DATABASE_URL` production để chạy smoke test hoặc restore.
3. Không dán password, token, private key hay dump vào Git.
4. Lệnh xoá phải có target cụ thể; không dùng biến chưa được kiểm tra.
5. Một backup chỉ đáng tin sau khi `pg_restore --list` và restore thử thành công.

Kiến thức trong sách không phụ thuộc việc bạn chọn native HTTP, Express hay NestJS. Framework thay đổi syntax; các invariant về identity, ownership, transaction, release và recovery vẫn giữ nguyên.

Hint đúc kết

- Đọc hệ thống theo luồng dữ liệu, không theo thứ tự thư mục.
- Luôn hỏi “ai sở hữu dữ liệu này?” trước khi viết query.
- Luôn hỏi “nếu bước này hỏng giữa chừng?” trước khi tự động hoá.
- Cài ít công cụ hơn nhưng hiểu rõ failure mode của từng công cụ.

1 Mental model của một hệ thống web

1.1 Mục tiêu

Sau chương này, bạn có thể vẽ lại đường đi của một request, phân biệt source với runtime và biết kiểm tra ở lớp nào khi người dùng báo “ứng dụng không chạy”.

1.2 Bảy khái niệm đủ để bắt đầu

- **Client** tạo HTTP request; trong dự án này là JavaScript chạy trong browser.
- **Server** nhận request và tạo response; đây là process Node.js.
- **Process** là một chương trình đang chạy, có PID, memory và lifecycle.
- **Port** giúp hệ điều hành chuyển network traffic tới đúng process.
- **API** là hợp đồng giữa client và server: method, path, input, status, output.
- **Database** giữ state lâu dài qua các lần restart.
- **Environment** cung cấp cấu hình thay đổi theo nơi chạy như port và URL DB.

Một mô hình tối thiểu:

```
Browser
-> HTTPS :443
-> Nginx reverse proxy
-> HTTP 127.0.0.1:5050
-> Node.js request handler
-> PostgreSQL 127.0.0.1:5432
<- JSON/HTML response
```

Nginx và Node đều là server nhưng ở hai boundary khác nhau. Nginx kết thúc TLS, giới hạn request và proxy. Node hiểu domain rule. PostgreSQL thực thi constraint, transaction và lưu state. Đặt đúng trách nhiệm giúp lỗi dễ cô lập.

1.3 Source, build artifact và runtime

`src/` là thứ con người sửa. `scripts/build.js` dùng `esbuild` tạo `dist/`. Production chạy `dist/server.js`, không chạy TypeScript từ `src/`.

```
source + dependencies + build configuration
      |
      v
    reproducible build
      |
      v
dist/server.js + dist/public + migrations
      |
      v
    production process
```

Điểm quan trọng là **build một lần, chạy cùng artifact**. Nếu mỗi host tự build theo dependency và tool version khác nhau, “cùng commit” chưa chắc là cùng phần mềm.

1.4 Theo dấu một request thật

Với GET /api/lessons, đường đi khái quát là:

1. src/index.ts nhận request, gán security headers và đo thời gian.
2. src/modules/request-context.ts đọc cookie, tìm user và tạo request ID.
3. src/modules/index.ts tập hợp endpoint registry.
4. src/lib/dispatcher.ts khớp method/path và kiểm tra access metadata.
5. Controller chuyển HTTP input thành lời gọi domain/repository.
6. Repository gọi src/lib/database.js bằng parameterized SQL.
7. Response đi ngược lại; logger ghi status và duration.

Nếu path đúng nhưng method sai, dispatcher trả 405 và header Allow. Nếu không đăng nhập ở endpoint user, nó trả 401; user thường gọi endpoint admin nhận 403. Đây là hợp đồng, không phải chi tiết UI.

Hệ thống hiện có

Endpoint được khai báo bằng { method, pattern, access, controller }. Access có ba mức public, user, admin. Unknown /api/* luôn trả JSON 404, không rơi về HTML của frontend.

1.5 Liveness, readiness và shutdown

/health/live trả lời “process còn phục vụ HTTP không?”. /health/ready còn query SELECT 1, nên trả lời “process có đủ dependency để nhận traffic không?”. Không trộn hai khái niệm: database tạm down không nhất thiết có nghĩa process đã chết.

Khi nhận SIGTERM, server dừng nhận kết nối mới, đóng connection còn lại sau timeout rồi đóng DB pool. Đây là graceful shutdown. Nếu gọi kill -9, ứng dụng không có cơ hội hoàn tất quy trình đó.

1.6 Lab: truy dấu endpoint

Lab thực hành

Preflight: đứng tại root repository; lab chỉ đọc source.

```
rg -n 'pattern: "/health/ready"' src/modules
rg -n 'handleReadiness|SELECT 1' src/modules
rg -n 'dispatchEndpoint' src
rg -n 'createRequestContext' src
```

Với mỗi kết quả, ghi lại input và output của tầng đó. Tiếp tục với một endpoint có access: "user" và tìm query cuối cùng.

Checkpoint

Bạn phải chỉ ra được nơi path được match, nơi access được chặn và nơi PostgreSQL được gọi. Nếu chưa, dừng bắt đầu sửa endpoint.

1.7 Failure modes

Triệu chứng	Lớp kiểm tra đầu tiên
Domain không mở	DNS, firewall, Nginx
502 Bad Gateway	Node process/port
/live 200, /ready 500	PostgreSQL/config
API 401	Cookie/session
API 403	Role/ownership
HTML cũ sau deploy	Artifact/cache/release symlink

1.8 Tự kiểm tra

1. Tại sao port 5050 không nên mở ra Internet khi đã có Nginx?
2. Tại sao source chạy được không chứng minh artifact production chạy được?
3. Một database outage nên làm liveness hay readiness thất bại?

Hint đúc kết

- Tìm entrypoint, request flow, data store và process manager trước khi đọc chi tiết.
- Dùng request ID để nối log giữa các boundary.
- Health check phải kiểm tra đúng thứ bạn định tự động phục hồi.
- Một lỗi được khoanh vùng tốt có giá trị hơn một lần restart may mắn.

Tiếp theo: [Thiết kế hệ thống nhiều người dùng](#).

2 Thiết kế hệ thống nhiều người dùng

2.1 Mục tiêu

Thiết kế multi-user không bắt đầu bằng màn hình login. Nó bắt đầu bằng việc xác định identity nào đang thao tác và record nào identity đó được phép chạm tới.

2.2 Bốn câu hỏi khác nhau

Khái niệm	Câu hỏi
Identity	Chủ thể này là ai?
Authentication	Bằng chứng nhận dạng có hợp lệ không?
Authorization	Chủ thể được phép thực hiện action nào?
Ownership	Record cụ thể này thuộc về ai?

Role `USER` cho phép gọi endpoint tạo note không đồng nghĩa user được sửa mọi note. Role trả lời quyền theo loại hành động; ownership trả lời quyền trên từng record.

2.3 Phân loại dữ liệu trước khi viết API

Dự án dùng ba loại chính:

- `USER_PRIVATE`: input, note, highlight, favorite, selection, study action.
- `ADMIN_SHARED`: lesson/catalog được admin quản lý và learner cùng đọc.
- `SYSTEM_CONFIG`: cấu hình build/runtime, không phải nội dung cá nhân.

Mỗi table private có `user_id` trực tiếp, hoặc join tới một record có `user_id`. Điều này làm ownership trở thành dữ liệu có thể query và test, thay vì một giả định trong UI.

```
SELECT id, note_date, content
FROM manual_notes
WHERE user_id = $1
ORDER BY note_date DESC;
```

Không query theo `id` rồi mới hy vọng controller nhớ so sánh owner. Cách an toàn hơn là đưa owner vào điều kiện tìm record:

```
DELETE FROM manual_notes
WHERE id = $1 AND user_id = $2
RETURNING id;
```

Nếu không có row trả về, API không cần tiết lộ record không tồn tại hay thuộc user khác.

2.4 IDOR và horizontal privilege escalation

IDOR xảy ra khi client thay ID trong URL/body và server tin rằng ID hợp lệ đồng nghĩa được phép truy cập. Đây là lỗi phía server; ấn nút trên frontend không sửa được.

Ví dụ nguy hiểm:

```
SELECT * FROM manual_notes WHERE id = $1;
```

User B lấy được UUID của User A bằng log, link chia sẻ nhầm hoặc đoán từ API khác thì có thể đọc record. UUID làm việc đoán khó hơn, không thay thế authorization.

Hệ thống hiện có

Các repository private trong dự án lọc theo authenticated `user_id`. Image bytes được join qua `user_inputs.user_id` trước khi trả nội dung. Smoke suite tạo hai user để kiểm tra B không đọc hoặc sửa dữ liệu của A.

2.5 Role hay permission?

Hai role `USER/ADMIN` đủ khi policy đơn giản và ổn định. Khi xuất hiện editor, support, content reviewer hoặc quyền theo tổ chức, một chuỗi `if role === ...` sẽ nhanh chóng khó kiểm soát.

Chuyển sang permission khi:

- Một user có nhiều vai trò theo tenant/project.
- Cùng action nhưng scope khác nhau.
- Policy cần audit và thay đổi không deploy code.

Đừng thiết kế RBAC tổng quát trước khi có yêu cầu. Với ứng dụng nhỏ, endpoint access metadata cộng ownership query rõ ràng dễ review hơn.

2.6 Lab: ma trận hai user

Lab thực hành

Preflight: chạy server với một `TEST_DATABASE_URL` có tên chứa `test`; không dùng dữ liệu production.

1. Đăng ký User A và User B, mỗi user giữ cookie jar riêng.
2. A tạo một manual note, ghi lại ID.
3. B thử list, read, update và delete ID của A.
4. Kỳ vọng B không thấy record và không thay đổi được nó.
5. A đọc lại record để chứng minh dữ liệu còn nguyên.

Có thể dùng smoke suite hiện tại làm oracle:

```
TEST_DATABASE_URL=postgresql://localhost:5432/ai_learn_jlpt_test \
npm run test:integration
```

Checkpoint

Test phải chứng minh cả **không lộ dữ liệu** và **không làm thay đổi dữ liệu**. Chỉ test response status là chưa đủ; đọc lại record của A sau phép thử của B.

2.7 Failure modes

- List lọc owner nhưng update/delete quên lọc.
- Table con không có `user_id`, join ownership sai hoặc thiếu.
- Endpoint admin được bảo vệ nhưng static CMS page lại bị coi là lớp bảo mật.
- Dùng email từ request body thay vì user ID từ authenticated context.
- Batch query nhận một mảng ID nhưng không giới hạn tất cả ID theo owner.

2.8 Tự kiểm tra

1. Vì sao UUID không phải access control?
2. Khi nào 404 an toàn hơn việc nói “record thuộc user khác”?
3. Shared catalog khác private note ở invariant nào?

Hint đúc kết

- Gắn owner vào record ngay từ lúc tạo.
- Lấy owner từ session context, không lấy từ payload của client.
- Viết ma trận quyền trước endpoint: anonymous, owner, non-owner, admin.
- Test mọi verb; lỗi ownership thường nằm ở update/delete hơn list.

Tiếp theo: [Xây dựng session authentication](#).

3 Xây dựng session authentication

3.1 Mục tiêu

Hiểu trọn vòng đời register/login/session/logout và các quyết định bảo mật đang được hiện thực trong `src/modules/common/auth/`.

3.2 Authentication flow

```
email + password
-> normalize + validate
-> lookup password hash
-> scrypt + timing-safe comparison
-> random session token
-> store SHA-256(token) in PostgreSQL
-> send raw token once in HttpOnly cookie
```

Server cần raw token để gửi client nhưng không cần lưu nó. Những request sau gửi cookie; server hash token rồi join `sessions` với `users`. Nếu database bị đọc trộm, token hash không thể được dùng trực tiếp làm cookie.

3.3 Password hash không phải encryption

Encryption có key để giải mã. Password storage cần hàm một chiều, chậm và có salt riêng. Dự án tạo salt 16 byte và dùng `crypto.scryptSync`; chuỗi lưu có dạng:

```
scrypt:<salt-hex>:<derived-key-hex>
```

Salt chống bảng hash dựng sẵn và làm hai user cùng password có hash khác nhau. `timingSafeEqual` giảm rò rỉ timing khi so sánh derived key.

Cảnh báo

Không tự thiết kế thuật toán mật mã. Việc định dạng hash có version/scheme là hữu ích để sau này rehash, nhưng primitive phải đến từ thư viện chuẩn đã review.

3.4 Session và cookie

Token được tạo bằng `randomBytes(32).toString("base64url")`. Session hết hạn sau 14 ngày. Cookie có:

- `HttpOnly`: JavaScript browser không đọc token.
- `SameSite=Lax`: giảm một số cross-site request.
- `Secure` ở production: chỉ gửi qua HTTPS.
- `Path=/:` dùng cho toàn ứng dụng.
- `Expires`: browser biết thời điểm bỏ cookie.

`HttpOnly` không chống việc browser tự gửi cookie trong request xấu; vì vậy XSS, CSRF và validation vẫn cần được xử lý riêng.

3.5 Request context và access

Mỗi request đọc token từ cookie, tìm session còn hạn và tạo `currentUser` công khai chỉ gồm `id, email, role`. Password hash không đi vào context hoặc response.

- Không có user ở endpoint private: 401 `Authentication required`.
- Có user nhưng thiếu role: 403 `Admin access required`.
- Có role nhưng record thuộc người khác: repository không trả record.

Logout xóa session trong database rồi gửi cookie có `Max-Age=0`. Chỉ xóa cookie mà không xóa DB session sẽ để token bị đánh cắp tiếp tục hợp lệ.

3.6 Session hay JWT?

Tiêu chí	Session DB	JWT access token
Revoke tức thì	Dễ, xóa row	Khó nếu không có denylist/TTL ngắn
Mỗi request	Query/cache session	Verify chữ ký
State server	Có	Thường không
Thay đổi role tức thì	Có	Chờ token mới
Phù hợp browser app nhỏ	Rất phù hợp	Thường không cần

JWT hữu ích giữa service hoặc hệ identity phân tán, nhưng không tự động an toàn hơn. Nếu cuối cùng vẫn cần revoke store, refresh token và rotation, độ phức tạp có thể cao hơn session.

3.7 Lab: auth bằng `curl`

Lab thực hành

Preflight: server local chạy trên test DB; file cookie nằm ngoài Git.


```

case "$DATABASE_URL" in
    *lab*|*test*) ;;
    *) echo 'Refusing non-test database'; exit 1 ;;
esac
cookie_jar="$(mktemp)"
base_url="http://127.0.0.1:5050"

curl -i -c "$cookie_jar" -H 'Content-Type: application/json' \
    -d
    ↪ '{"email":"cookbook-user@example.test","password":"safe-lab-password"}'
    ↪ \
    "$base_url/api/auth/register"

curl -sS -b "$cookie_jar" "$base_url/api/me"
curl -i -b "$cookie_jar" -c "$cookie_jar" -X POST \
    "$base_url/api/auth/logout"
curl -sS -b "$cookie_jar" "$base_url/api/me"

psql "$DATABASE_URL" -Xv ON_ERROR_STOP=1 -c \
    "DELETE FROM users WHERE email='cookbook-user@example.test'"
rm -f "$cookie_jar"

```

Kỳ vọng: register trả 201 và Set-Cookie; /me đầu có user; logout trả 200; /me cuối có user: null.

Checkpoint

Mở cookie jar chỉ để xác nhận attribute/format, không chép token vào log hoặc tài liệu. Query table `sessions` phải thấy token hash, không thấy raw cookie token.

3.8 Failure modes

- Trả password hash trong JSON do dùng `SELECT *`.
- Phân biệt “email không tồn tại” và “password sai”, hỗ trợ user enumeration.
- Production không dùng HTTPS khiến cookie `Secure` không hoạt động đúng luồng.
- Session hết hạn không được dọn định kỳ.
- Role thay đổi nhưng session/JWT cũ vẫn mang policy cũ.

3.9 Tự kiểm tra

1. Tại sao salt có thể lưu cạnh hash?
2. Token hash bảo vệ tình huống nào?
3. Khác biệt giữa xoá cookie và revoke session là gì?

Hint đúc kết

- Auth response chỉ trả public user projection.
- Dùng thông báo login chung để giảm user enumeration.
- Session ID là credential; bảo vệ như password.
- Chọn session trước nếu hệ thống là browser app cùng domain và cần revoke dễ.

Tiếp theo: [Làm auth đủ an toàn cho production.](#)

4 Làm auth đủ an toàn cho production

4.1 Mục tiêu

Auth hiện tại có nền tảng tốt nhưng “đăng nhập được” khác “chịu được Internet”. Chương này sắp thứ tự các lớp phòng vệ cần bổ sung theo rủi ro.

4.2 Threat model tối thiểu

Giả định attacker có thể:

- Gửi login/register tự động với tốc độ cao.
- Biết email từ nguồn khác và thử password bị lộ.
- Lừa user mở trang ngoài để gửi request chéo.
- Có được một session token qua máy dùng chung hoặc log sai cách.
- Thay ID, role hoặc `user_id` trong payload.

Threat model biến từ “hãy làm auth an toàn” thành test cụ thể.

4.3 Ưu tiên 1: giới hạn thử credential

Khuyến nghị

Thêm rate limit theo cả IP và normalized account key. Ví dụ: cửa sổ ngắn cho IP, backoff tăng dần cho account, log sự kiện nhưng không log password. Trả response giống nhau cho email tồn tại và không tồn tại.

Không khoá account vĩnh viễn sau vài lần sai; attacker có thể dùng cơ chế đó để DoS người khác. Một thiết kế hợp lý kết hợp delay, limit và cảnh báo.

4.4 Ưu tiên 2: session lifecycle

- Rotate session sau login hoặc thay đổi privilege.
- Cho user xem và revoke các session khác.
- Revoke toàn bộ khi đổi/reset password.
- Dọn `expires_at <= NOW()` bằng job định kỳ.
- Lưu metadata tối thiểu phục vụ audit như created time; cân nhắc privacy trước khi lưu IP/user agent dài hạn.

Query cleanup đơn giản:

```
DELETE FROM sessions WHERE expires_at <= NOW();
```

4.5 CSRF và SameSite

SameSite=Lax giảm nhiều cross-site POST truyền thống nhưng không thay thế toàn bộ CSRF defense. Nếu có endpoint nhạy cảm, cross-origin integration hoặc thay đổi cookie policy, dùng CSRF token gắn với session và kiểm tra Origin.

Không dùng GET để thay đổi state. GET /logout hay GET /delete khiến browser, prefetcher và crawler có thể kích hoạt action ngoài ý muốn.

4.6 Email verification và reset password

Hai luồng này nên dùng token ngẫu nhiên một lần, lưu token hash cùng expiry:

```
request reset
-> luôn trả response chung
-> nếu account tồn tại: tạo random token
-> lưu hash(token), purpose, user_id, expires_at, used_at
-> gửi raw token qua email
-> consume trong transaction
-> đổi password + revoke sessions + đánh dấu used_at
```

Schema đề xuất có thể dùng table auth_tokens với unique token hash, purpose giới hạn VERIFY_EMAIL/RESET_PASSWORD, expiry ngắn và used_at.

Cảnh báo

Không lưu raw reset token, không gửi password mới qua email và không cho phép một token được dùng hai lần. Link reset có thể xuất hiện trong browser history hoặc proxy log; TTL phải ngắn.

4.7 Audit, MFA và OAuth

Audit log nên ghi event, actor_user_id, target_user_id, request ID, time và outcome; không ghi password/token/body. Sự kiện quan trọng: login success/fail, password reset, role change, revoke session và admin action.

MFA đáng thêm khi account có dữ liệu giá trị cao hoặc quyền admin. OAuth/OIDC đáng thêm khi tổ chức đã có identity provider. Cả hai tạo recovery và operational work mới; không nên thêm chỉ để thay form login đang đủ dùng.

4.8 Recipe triển khai theo lát dọc

1. Viết threat/test case trước.
2. Thêm schema bằng migration forward-only.
3. Tạo repository chỉ xử lý token hash và transaction.

4. Tạo service áp policy TTL/one-time/revoke.
5. Controller chỉ parse input và map lỗi chung.
6. Thêm audit event và metric.
7. Test token hết hạn, reuse, concurrent consume và cross-user.

4.9 Lab: review auth hiện tại

Lab thực hành

Không sửa code. Lập bảng cho `register`, `login`, `logout`, `change role` với các cột: asset, attacker, abuse case, control hiện có, control còn thiếu, test.
 Đối chiếu các file `auth.js`, `session.js`, `controller.ts`, `dispatcher.ts` và `smoke test`.
 Ưu tiên ba gap có impact cao nhất thay vì liệt kê công nghệ.

4.10 Failure modes

- Rate limit chỉ theo IP, làm hỏng user sau NAT hoặc dễ đổi IP để vượt.
- Reset token không one-time hoặc không revoke session cũ.
- Audit log chứa credential.
- Bật OAuth nhưng bỏ qua account linking takeover.
- Thêm MFA nhưng không thiết kế recovery code/support flow.

4.11 Tự kiểm tra

1. Tại sao account lock cứng có thể trở thành DoS?
2. Vì sao reset password cần revoke session?
3. Khi nào SameSite chưa đủ chống CSRF?

Hint đúc kết

- Hardening theo threat, không theo checklist công nghệ.
- Token một lần phải có purpose, expiry và atomic consume.
- Admin là mục tiêu ưu tiên cho MFA và audit.
- Recovery flow thường rủi ro hơn happy-path login; review nó trước go-live.

Tiếp theo: [Cài đặt và kết nối PostgreSQL](#).

5 Cài đặt và kết nối PostgreSQL

5.1 Mục tiêu

Hiểu PostgreSQL đang chạy ở đâu, đăng nhập bằng identity nào và phân biệt lỗi service, network, role với database. Đây là nền để mọi chương SQL sau có thể thực hành an toàn.

5.2 Mental model

```
PostgreSQL cluster (một server process)
+-- role: postgres          quản trị
+-- role: ai_learn_app      ứng dụng
+-- database: postgres      maintenance
+-- database: ai_learn_jlpt
    +-- schema: public
        +-- users
        +-- sessions
        +-- ...
```

Role là identity/quyền trong PostgreSQL. **Database** là vùng dữ liệu có connection boundary. **Schema** là namespace bên trong database. **Table** giữ row. Một cluster có nhiều database; một connection chỉ làm việc trong một database tại một thời điểm.

5.3 Cài cho development

Trên macOS với Homebrew:

```
brew install postgresql@16
brew services start postgresql@16
psql --version
pg_isready
```

Trên Ubuntu VPS:

```
sudo apt update
sudo apt install postgresql postgresql-client
sudo systemctl enable --now postgresql
sudo systemctl status postgresql --no-pager
```

Docker phù hợp cho môi trường lab tạm thời nhưng thêm volume/network lifecycle. Nếu đã có PostgreSQL native ổn định, không cần thêm container chỉ để “chuẩn hoá”.

5.4 Tạo role và database

Production nên dùng role riêng, không để ứng dụng kết nối bằng superuser:

```
sudo -u postgres createuser --no-superuser --no-createdb --no-createrole
↪ ai_learn_app
sudo -u postgres createdb --owner=ai_learn_app ai_learn_jlpt
sudo -u postgres psql -c "ALTER ROLE ai_learn_app PASSWORD 'REPLACE_ME';"
```

Không đặt password thật trực tiếp trong shell history. Script provision của dự án tạo password ngẫu nhiên và ghi environment file quyền hạn chế.

Connection string có cấu trúc:

```
postgresql://USER:PASSWORD@HOST:PORT/DATABASE
```

Ví dụ local không có credential thật:

```
DATABASE_URL=postgresql://localhost:5432/ai_learn_jlpt
```

Hệ thống hiện có

src/lib/config.js fail fast nếu thiếu DATABASE_URL; src/lib/database.js tạo pg.Pool. Server query SELECT 1 trước khi listen nên cấu hình DB sai không tạo ra một process “có vẻ chạy” nhưng vô dụng.

5.5 Làm việc với psql

```
psql "$DATABASE_URL"
```

Các meta-command chạy bên trong psql:

Lệnh	Ý nghĩa
\conninfo	Connection hiện tại
\l	Danh sách database
\du	Role và attribute
\c db_name	Đổi database
\dn	Danh sách schema
\dt	Table trong search path
\d users	Mô tả table/index/constraint
\x	Bật/tắt expanded output
\timing	Hiện thời gian query
\q	Thoát

Với automation, dùng `-x` để không đọc `.psqlrc`, `-A -t` để output máy đọc và `-v ON_ERROR_STOP=1` để dừng khi SQL lỗi.

5.6 Lab: database cô lập

Lab thực hành

Preflight: xác nhận đang ở máy local và user có quyền tạo database.

```
createdb ai_learn_jlpt_lab
export LAB_DATABASE_URL=postgresql://localhost:5432/ai_learn_jlpt_lab
psql "$LAB_DATABASE_URL" -Xv ON_ERROR_STOP=1 -c \
    "SELECT current_database(), current_user, version();"

```

Apply schema của dự án:

```
DATABASE_URL="$LAB_DATABASE_URL" npm run db:migrate
psql "$LAB_DATABASE_URL" -Xc '\dt'
```

Cleanup sau khi thoát mọi connection:

```
dropdb ai_learn_jlpt_lab
unset LAB_DATABASE_URL
```

Cảnh báo

Trước `dropdb`, in chính xác target: `printf '%s\n' "$LAB_DATABASE_URL"`. Script tự động nên từ chối nếu tên database không chứa `lab` hoặc `test`.

5.7 Chẩn đoán theo lớp

Lỗi	Kiểm tra
connection refused	Service, host, port, firewall
password authentication failed	Role/password/pg_hba.conf
database does not exist	Tên DB và cluster đang kết nối
permission denied	Owner/grant; không giải bằng superuser lâu dài
relation does not exist	Database/schema/search path/migration

5.8 Tự kiểm tra

1. Role khác database thế nào?

2. Vì sao application role không nên có `CREATEDB` hoặc superuser?
3. `pg_isready` chứng minh được gì và chưa chứng minh được gì?

Hint đúc kết

- Luôn bắt đầu bằng `\conninfo`; nhiều lỗi SQL thực ra là kết nối nhầm database.
- Tách credential của app, migration và DBA khi hệ thống lớn dần.
- Connection URL là secret nếu chứa password.
- Production database không cần public port khi app chạy cùng private network.

Tiếp theo: [SQL thực chiến](#).

6 SQL thực chiến trên schema dự án

6.1 Mục tiêu

Đọc và viết SQL bảo toàn invariant, chống injection và có thể giải thích bằng query plan. Ví dụ dùng schema AI Learn JLPT thay vì table minh hoạ xa lạ.

6.2 Data type là một phần của thiết kế

- **UUID**: ID khó va chạm, tạo độc lập; không phải access control.
- **TIMESTAMPZ**: thời điểm tuyệt đối; hiển thị theo timezone ở UI.
- **DATE**: ngày lịch như ngày học, không phải timestamp lúc nửa đêm.
- **JSONB**: cấu trúc linh hoạt cần query; không thay thế mọi column/relationship.
- **BYTEA**: binary nhỏ/vừa cần transaction cùng dữ liệu; ảnh lớn nên cân nhắc object storage khi scale.

NOT NULL, DEFAULT, CHECK, UNIQUE và foreign key làm database bảo vệ invariant dù dữ liệu đi vào từ API, migration hay admin script.

6.3 CRUD có scope

Insert một note private:

```
INSERT INTO manual_notes (user_id, note_date, content)
VALUES ($1, $2, $3)
RETURNING id, note_date, content, completed;
```

Update phải khóa theo owner:

```
UPDATE manual_notes
SET content = $3, updated_at = NOW()
WHERE id = $1 AND user_id = $2
RETURNING id, content, updated_at;
```

Placeholder \$1 không chỉ giúp escape quote; nó tách SQL structure khỏi data. Không thể parameterize tên table/column — dynamic identifier cần allowlist.

6.4 JOIN và quan hệ

Session lookup hiện tại join identity với credential state:

```
SELECT users.id, users.email, users.role
FROM sessions
JOIN users ON users.id = sessions.user_id
WHERE sessions.token_hash = $1
AND sessions.expires_at > NOW();
```

Foreign key `sessions.user_id REFERENCES users(id) ON DELETE CASCADE` đảm bảo xoá user cũng xoá session. `CASCADE` hữu ích khi child không còn ý nghĩa độc lập; với lịch sử/audit, có thể cần `RESTRICT` hoặc `SET NULL`.

6.5 Aggregate, CTE và pagination

Đếm study action theo ngày:

```
SELECT action_date, count(*) AS actions
FROM study_actions
WHERE user_id = $1 AND action_date >= $2
GROUP BY action_date
ORDER BY action_date;
```

CTE làm các bước biến đổi dễ đọc, nhưng đừng dùng nó để che một query quá phức tạp. Pagination nhỏ có thể dùng `LIMIT/OFFSET`; dataset lớn nên dùng cursor dựa trên cặp sort ổn định như `(created_at, id)`.

6.6 Transaction

Transaction cần khi nhiều câu lệnh phải cùng thành công hoặc cùng thất bại:

```
BEGIN;
UPDATE users SET password_hash = $1, updated_at = NOW() WHERE id = $2;
DELETE FROM sessions WHERE user_id = $2;
COMMIT;
```

Nếu update password thành công nhưng revoke session thất bại, invariant bảo mật bị phá. Transaction đóng gói hai thay đổi thành một đơn vị atomic.

Trong Node, mọi câu của transaction phải dùng **cùng connection**. Gọi helper `query()` trên pool cho từng câu có thể lấy connection khác nhau.

6.7 Index và query plan

Index đổi chi phí đọc lấy chi phí storage/write. Tạo index theo query thật:

```
CREATE INDEX idx_manual_notes_user_date
ON manual_notes(user_id, note_date);
```

Thứ tự column quan trọng. Index trên (user_id, note_date) phù hợp filter owner rồi range/order theo date. Nó không nhất thiết tốt cho query chỉ theo note_date.

Đọc plan:

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM manual_notes
WHERE user_id = '00000000-0000-0000-0000-000000000000'
ORDER BY note_date DESC
LIMIT 20;
```

ANALYZE thực thi query; không dùng tùy tiện với DELETE/UPDATE production. Quan sát estimated/actual rows, scan type, sort và buffer — đừng chỉ tìm chữ “Index Scan”. Table nhỏ thường sequential scan là đúng.

6.8 JSONB: dùng có chủ đích

metadata JSONB phù hợp thuộc tính phụ thay đổi theo action. Những field cần foreign key, unique, range validation hoặc join thường nên là column. Nếu query một JSON path thường xuyên, thêm expression/GIN index sau khi đo workload.

6.9 Lab: query an toàn

Lab thực hành

Preflight: LAB_DATABASE_URL phải chứa lab hoặc test.

```
case "$LAB_DATABASE_URL" in
  *lab*|*test*) ;;
  *) echo 'Refusing non-lab database'; exit 1 ;;
esac
psql "$LAB_DATABASE_URL" -Xv ON_ERROR_STOP=1
```

Trong psql:

```

\timing on
BEGIN;
INSERT INTO users (email, password_hash)
VALUES ('sql-lab@example.test', 'lab:not-a-real-password-hash')
RETURNING id;
ROLLBACK;

```

Sau rollback, query email phải trả 0 row. Tiếp theo chọn một index hiện có trong `\d manual_notes`, viết query phù hợp và so sánh `EXPLAIN` trước/sau khi tăng dữ liệu lab.

Checkpoint

Bạn phải giải thích được constraint nào bảo vệ dữ liệu, query nào dùng index và tại sao transaction rollback không để lại row.

6.10 Failure modes

- `SELECT *` làm API vô tình lộ column nhạy cảm.
- Dùng offset lớn gây scan/sort tốn kém.
- Index từng column nhưng query cần composite index.
- Transaction giữ mở trong lúc gọi network API, gây lock lâu.
- JSONB biến thành nơi cất mọi thứ và mất constraint.

6.11 Tự kiểm tra

1. Khi nào `ON DELETE CASCADE` không phù hợp?
2. Vì sao transaction trong pool cần cùng client?
3. Tại sao sequential scan chưa chắc là lỗi?

Hint đúc kết

- Viết invariant trước, rồi chọn constraint và transaction để bảo vệ nó.
- Index theo filter + sort của query nóng, không theo cảm giác.
- Luôn scope private query bằng owner.
- Đo plan trên dữ liệu gần với production; table 10 row đánh lừa optimizer review.

Tiếp theo: PostgreSQL bên trong ứng dụng.

7 PostgreSQL bên trong ứng dụng

7.1 Mục tiêu

Kết nối kiến thức SQL với lifecycle của Node process: pool, transaction, migration, backup và các failure mode chỉ xuất hiện khi có concurrency.

7.2 Connection pool

Mở connection PostgreSQL có chi phí; giữ một pool cho process giúp tái sử dụng. `src/lib/database.js` lazy-create `pg.Pool` với:

- `max`: số connection tối đa của process.
- `connectionTimeoutMillis`: chờ mở/lấy connection bao lâu.
- `idleTimeoutMillis`: đóng connection idle sau bao lâu.

Pool sizing là bài toán toàn hệ thống:

```
tổng connection tiềm năng
= số app process × DB_POOL_MAX
+ migration/admin/monitoring headroom
```

Nếu PostgreSQL cho 100 connection, 10 replica × pool 20 là cấu hình không thể đạt. Pool không tạo capacity; nó giới hạn concurrency vào database.

7.3 Query và transaction

Helper `query(text, params)` phù hợp câu độc lập. `withClient(callback)` giữ một client cho transaction:

```
await withClient(async (client) => {
  await client.query("BEGIN");
  try {
    await client.query("UPDATE ...", values);
    await client.query("DELETE ...", values);
    await client.query("COMMIT");
  } catch (error) {
    await client.query("ROLLBACK");
    throw error;
  }
});
```

Luôn release client trong `finally`. Client bị giữ quên sẽ làm pool cạn dần và request mới timeout dù database vẫn healthy.

7.4 Migration như release contract

Migration runner đọc file `NNN_name.sql`, sort, kiểm tra `schema_migrations` rồi chạy mỗi file trong transaction. Quy tắc:

1. Migration đã apply không được sửa nội dung trong lịch sử.
2. Thay đổi mới là file số tiếp theo.
3. Migration phải tương thích với version app trong cửa sổ rollout/rollback.
4. Backup trước migration có rủi ro dữ liệu.
5. Chạy migration một lần ở release step, không để mỗi replica tranh nhau.

Forward-only không có nghĩa không thể phục hồi. Nó nghĩa rollback thường là một migration sửa tiếp hoặc restore có kiểm soát, không phải tự động chạy “down.sql” trên production đang có traffic.

7.5 Lock, deadlock và timeout

Update giữ row lock tới cuối transaction. Schema migration có thể cần lock mạnh hơn. Một transaction dài làm request khác chờ; hai transaction lấy resource theo thứ tự ngược nhau có thể deadlock và PostgreSQL sẽ huỷ một transaction.

Giảm rủi ro bằng cách:

- Transaction ngắn, không gọi HTTP/email bên trong.
- Cập nhật resource theo thứ tự ổn định.
- Đặt lock/statement timeout phù hợp cho migration.
- Retry chỉ transaction idempotent với lỗi transient đã phân loại.

7.6 Backup và restore

Custom format hỗ trợ validate và chọn object khi restore:

```
pg_dump -Fc --no-owner --no-acl "$DATABASE_URL" -f app.dump
pg_restore --list app.dump >/dev/null
shasum -a 256 app.dump > app.dump.sha256
```

Restore an toàn vào database mới:

```
createdb ai_learn_jlpt_restore_test
pg_restore --exit-on-error --no-owner --no-acl \
  --dbname=postgresql://localhost:5432/ai_learn_jlpt_restore_test app.dump
```

Sau restore: chạy migration mới nhất, smoke test và kiểm tra ownership/content. Chỉ đổi `DATABASE_URL` sau khi database mới đạt acceptance criteria.

Hệ thống hiện có

Các script trong `scripts/ops/` tạo dump, validate bằng `pg_restore --list`, ghi SHA-256 và từ chối initial restore nếu database đã có table. Release script tạo backup predeploy trước migration.

7.7 Lab: recovery drill

Lab thực hành

Preflight: dùng source database lab và tên target chứa `restore_test`.

1. Tạo vài record marker trong DB lab.
2. Chạy `pg_dump -Fc` và `pg_restore --list`.
3. Restore vào database mới.
4. So sánh migration count, user count và marker record.
5. Chạy application smoke test với target restore.
6. Drop cả hai DB lab sau khi ghi kết quả.

Checkpoint

Một file dump tồn tại không phải checkpoint. Checkpoint là ứng dụng kết nối được, migration state đúng và các invariant dữ liệu vượt qua smoke test.

7.8 Failure modes

- Pool max hợp lý cho một process nhưng vượt DB limit khi scale replica.
- Migration sửa table lớn giữ lock lâu hơn thời gian deploy.
- Backup nằm cùng disk/VPS và mất cùng incident.
- Restore trực tiếp đề production thay vì tạo database mới.
- Retry query ghi dữ liệu không idempotent tạo duplicate.

7.9 Tự kiểm tra

1. Pool 10 với 8 replica cần reserve ít nhất bao nhiêu connection?
2. Tại sao backup và migration phải nằm trước switch traffic?
3. Restore test cần kiểm tra điều gì ngoài row count?

Hint đúc kết

- Budget connection ở cấp hệ thống, không ở từng service riêng lẻ.
- Migration là code production; review lock và backward compatibility.
- Recovery drill nên chạy định kỳ trước khi có incident.
- Giữ ít nhất một backup ngoài failure domain của VPS.

Tiếp theo: Ghép các thành phần thành hệ thống.

8 Ghép các thành phần thành hệ thống

8.1 Mục tiêu

Nhìn codebase như các boundary cộng tác thay vì một danh sách file. Mục tiêu của kiến trúc không phải tạo nhiều layer, mà là đặt mỗi quyết định ở đúng nơi.

8.2 Dependency map

```
Browser UI
|
HTTP controller + endpoint metadata
|
Domain service / use-case
|
Repository contract
|
PostgreSQL adapter

Cross-cutting: config, logger, auth context, error mapping, build
```

- Controller hiểu HTTP nhưng không nên chứa SQL dài.
- Service hiểu use-case nhưng không nên biết cookie/header.
- Repository hiểu persistence và ownership query.
- Composition root nối dependency và đăng ký endpoint.

Layer không cần là class hoặc framework. Trong dự án, TypeScript được dùng ở HTTP/composition boundary; service/repository JavaScript vẫn có public contract rõ. Boundary quan trọng hơn đuôi file.

8.3 Request flow và failure boundary

```
request
-> validate config/request
-> resolve identity
-> authorize endpoint
-> execute domain action
-> query transactionally
-> map result/error
-> structured log + response
```

Mỗi bước phải trả lời: input tin cậy đến đâu, error nào có thể xảy ra, log gì mà không lộ secret, và retry có an toàn không.

8.4 Config

Config thay đổi theo environment; code behavior/invariant nằm trong source. `HOST`, `PORT`, `DATABASE_URL`, pool size và timeout là config. SQL table name, cookie security policy production hay access rule không nên được biến thành một biến tùy tiện chỉ để “linh hoạt”.

Validate config trước listen. Fail fast tạo lỗi rõ ở deploy thay vì lỗi ngẫu nhiên ở request đầu tiên.

8.5 Error và log

Error nội bộ cần context để điều tra; response public cần ổn định và không lộ stack/SQL. Structured log của request nên có request ID, method, path, status, duration. Không log cookie, password, token hoặc toàn bộ request body.

Request ID giúp ghép `request_failed` với log proxy và client report. Chỉ chấp nhận request ID đầu vào có charset/length an toàn; nếu không, tự sinh UUID.

8.6 Build và static assets

Build script bundle backend, browser JS/CSS, copy HTML/config và migrations vào `dist/`. Artifact production phải chứa đủ:

- `dist/server.js`.
- `dist/public/`.
- `dist/bin/migrate.js`.
- `dist/migrations/`.
- Production dependencies theo lockfile.

Smoke test artifact tìm lỗi mà unit test source không thấy: asset thiếu, path resolution sai hoặc migration không được đóng gói.

8.7 Security và resilience là thành phần

Security headers, body limit, ownership filter và parameter binding không phải “cleanup sau”. Health check, timeout, shutdown, backup và restart cũng là một phần thiết kế. Một feature không hoàn thành nếu chỉ có happy path controller.

Hệ thống hiện có

Architecture tests ngăn import xuyên boundary, dispatcher thống nhất access và smoke suite kiểm tra auth, CMS, image ownership cùng nhiều loại private learning data. Production process chạy artifact `dist/`.

8.8 Lab: thiết kế endpoint trước khi code

Lab thực hành

Thiết kế `POST /api/manual-notes/:id/archive` trên giấy:

1. Endpoint metadata: method, pattern, access.
 2. Request context: identity nào được dùng.
 3. Validation: ID và state transition.
 4. Repository query có `id + user_id`.
 5. Response codes cho anonymous, non-owner, missing và success.
 6. Log fields, không log content.
 7. Test matrix và migration nếu cần column mới.
- Sau đó chỉ ra file/boundary sẽ thay đổi; không cần viết code.

Checkpoint

Thiết kế đạt khi reviewer có thể kiểm tra ownership và failure behavior mà không cần đoán controller sẽ “nhớ làm đúng”.

8.9 Failure modes

- Controller vừa parse HTTP, vừa query, vừa quyết định business rule.
- Shared module import internal repository của feature khác.
- Catch mọi error và trả 200 với `{error}`.
- Build production khác lệnh được CI kiểm tra.
- Health endpoint luôn 200 dù dependency bắt buộc đã chết.

8.10 Tự kiểm tra

1. Boundary nào nên biết cookie?
2. Tại sao migration phải nằm trong artifact?
3. Request ID giải quyết vấn đề quan sát nào?

Hint đúc kết

- Kiến trúc tốt làm policy dễ tìm và test, không tối đa số layer.
- Composition root là nơi dependency gặp nhau; domain không nên tự tìm dependency.
- Build artifact là sản phẩm deploy, không phải thư mục phụ.
- Hoàn thiện vertical slice gồm auth, data, error, test và operation.

Tiếp theo: [Chọn framework, OS và plugin](#).

9 Chọn framework, OS và plugin

9.1 Mục tiêu

Chọn công cụ theo constraint và operational cost. “Phổ biến” không có nghĩa phù hợp; dependency mới luôn tạo thêm update, security và failure surface.

9.2 Framework HTTP

Lựa chọn	Điểm mạnh	Chi phí	Dùng khi
Native Node HTTP	Ít dependency, hiểu rõ protocol	Tự xây routing/middleware	API nhỏ, contract ổn định
Express	Ecosystem lớn, quen thuộc	Middleware quality không đồng đều	Team cần convention phổ biến
Fastify	Validation/plugin model, hiệu năng	Học lifecycle/plugin	API tăng nhanh, cần schema
NestJS	DI/module/convention mạnh	Nặng và nhiều abstraction	Team lớn, domain/module phức tạp

Native HTTP hiện tại không phải thiếu framework; nó là một lựa chọn có chủ đích. Chuyển framework chỉ đáng giá nếu vấn đề lặp lại như validation, plugin lifecycle hoặc team convention lớn hơn migration cost.

9.3 Database access

Raw SQL với pg cho query rõ, tận dụng PostgreSQL và ít abstraction. Query builder hỗ trợ composition/type nhưng vẫn cần hiểu SQL. ORM tăng tốc CRUD và relation phổ biến nhưng không xoá nhu cầu hiểu transaction, index hoặc plan.

Chọn ORM khi productivity/schema tooling thắng chi phí generated query và migration convention. Không thêm ORM chỉ để tránh học SQL — người trực production vẫn phải đọc query và lock.

9.4 Process và packaging

- **systemd**: có sẵn trên Ubuntu, restart, log, sandbox, timer; phù hợp VPS.
- **PM2**: tiện cho Node cluster/UI nhưng trùng nhiều trách nhiệm với systemd.
- **Docker**: artifact và isolation nhất quán; thêm registry, image, volume, network và update lifecycle.

Một VPS đơn không cần đồng thời PM2, Docker và systemd quản lý cùng process. Trong case study, systemd chạy Node artifact là đường ngắn và dễ điều tra.

9.5 Reverse proxy và OS

Nginx mature, explicit và có hệ sinh thái lớn. Caddy đơn giản hoá automatic TLS. Nếu provision/runbook đã dùng Nginx ổn định, đổi proxy chỉ vì config ngắn hơn ít tạo giá trị.

Ubuntu/Debian có package/documentation phù hợp server phổ thông. Alpine nhỏ hơn nhưng musl và package khác có thể làm native dependency khó debug. Image nhỏ không tự động đồng nghĩa hệ thống rẻ hơn nếu thời gian vận hành tăng.

9.6 Khi nào thêm thành phần

Thành phần	Signal thực tế
Redis cache	Đã đo query nóng, cache semantics/invalidations rõ
Queue/worker	Công việc chậm cần retry, không nằm trong request latency
Object storage	Binary lớn, cần CDN/lifecycle/scale độc lập DB
External IdP	SSO doanh nghiệp, federation hoặc compliance
Multiple replicas	Availability/throughput đã vượt một process
Kubernetes	Nhiều service/team cần orchestration chuẩn, không phải một VPS

9.7 Dependency gate

Trước khi cài package/plugin, trả lời:

1. Vấn đề cụ thể và metric nào chứng minh?
2. Có thể giải bằng platform/20 dòng code rõ ràng không?
3. Package có maintainer, release và security policy còn hoạt động không?
4. Có bao nhiêu transitive dependency và quyền runtime nào?
5. Test/upgrade/rollback ai sở hữu?
6. Nếu package dừng, exit strategy là gì?

Hệ thống hiện có

Runtime dự án chỉ có dependency pg, esbuild, TypeScript và tsx là development dependencies. Nginx/systemd/PostgreSQL xử lý trách nhiệm hạ tầng thay vì thêm package Node tương đương.

9.8 Bộ cài tối thiểu

Development: Git, Node 22, npm, PostgreSQL client/server, editor. Pandoc và XeLaTeX chỉ cần nếu build sách.

VPS đơn: Node runtime, PostgreSQL, Nginx, Certbot, curl và systemd có sẵn. Không cài compiler/dev dependency sau khi artifact đã build.

CI: checkout, Node, npm; build/test/package rồi dùng OpenSSH có sẵn để deploy.

9.9 Lab: architecture decision record

Lab thực hành

Chọn một đề xuất như “thêm Redis” và viết ADR một trang: context, decision, alternatives, evidence, operational cost, rollback và trigger review lại. Nếu không có metric/constraint cụ thể, decision mặc định là chưa thêm.

9.10 Tự kiểm tra

1. Docker giải quyết vấn đề nào mà systemd deployment hiện tại chưa giải quyết?
2. ORM có loại bỏ nhu cầu migration review không?
3. Signal nào hợp lý để thêm queue?

Hint đúc kết

- Tối ưu tổng chi phí sở hữu, không tối ưu số dòng config.
- Ưu tiên primitive của OS/database trước dependency cùng chức năng.
- Một công cụ mới phải có owner, upgrade path và failure playbook.
- Trì hoãn dependency là quyết định kiến trúc hợp lệ khi evidence chưa đủ.

Tiếp theo: [Deploy và CI/CD](#).

10 Deploy và CI/CD

10.1 Mục tiêu

Hiểu release như một state transition có checkpoint, không phải thao tác copy file. Mỗi bước phải fail closed và để lại đường phục hồi.

10.2 Topology VPS đơn

```
DNS -> VPS :443 -> Nginx -> Node 127.0.0.1:5050 -> PostgreSQL
↔ localhost:5432

                        |                               |
                        + systemd restart               + daily backup

GitHub Actions -> SSH deploy user -> versioned release directories
```

Chỉ SSH/80/443 mở ở firewall. Node và PostgreSQL nghe loopback/private network. Nginx kết thúc TLS và giữ public boundary nhỏ.

10.3 Artifact và release directory

CI chạy `npm ci`, check, test và build. Artifact gồm `dist`, `manifest/lockfile` và ops scripts. Host giải nén vào:

```
/opt/ai-learn-jlpt/
  releases/<commit-sha>/
  current -> releases/<active-sha>/
  shared/
  incoming/
```

Versioned directory làm switch/rollback mã bằng symlink atomic. Không chép đè file process đang dùng; không `git pull` và build tùy ý trên production.

10.4 Thử tự release

```
verify artifact
-> acquire deploy lock
-> install production dependencies
-> backup database
-> apply forward migration
-> atomically switch current
-> restart service
-> wait /health/ready
-> keep or rollback application symlink
```

Backup trước migration vì migration là thay đổi state khó đảo. Switch sau migration vì app mới có thể cần schema mới. Health gate sau restart vì `systemctl restart` thành công chỉ chứng minh process được tạo, chưa chứng minh app dùng được.

Hệ thống hiện có

`.github/workflows/deploy-production.yml` serialize production workflow, build artifact rồi upload qua SSH. `remote-deploy.sh` dùng `flock`, backup predeploy, migrate, switch symlink, chờ readiness và rollback app nếu release mới không ready. `systemd` tự restart khi process crash; timer restart sau nhiều readiness failure liên tiếp.

10.5 Secret và least privilege

GitHub environment giữ host, port, user, private key, known hosts và production URL. `known_hosts` phải được đối chiếu fingerprint qua console tin cậy; nếu chỉ chạy `ssh-keyscan` trên cùng network chưa xác minh, ta vẫn có thể lưu key của attacker.

Deploy user sở hữu release directory nhưng `sudo` chỉ được start/stop/restart đúng service/timer. App bind loopback port nên không cần root. Environment file DB có quyền đọc hạn chế và không nằm trong artifact.

10.6 systemd và Nginx

`systemd` định nghĩa command, user, working directory, environment, restart và sandbox. Nginx proxy Host, client IP/proto và giới hạn body/timeout. Certbot quản lý certificate lifecycle.

Cookie Secure phụ thuộc HTTPS end-to-end từ browser tới Nginx. Node có thể nhận HTTP loopback nhưng public URL phải là HTTPS.

10.7 Rollback nào?

Rollback application đổi symlink về artifact cũ. Migration forward đã chạy vẫn còn đó. Vì vậy migration phải backward-compatible trong cửa sổ rollback hoặc cần recovery procedure

riêng.

Pattern expand/contract:

1. Expand schema, app cũ và mới cùng chạy được.
2. Deploy app dùng schema mới.
3. Backfill/quan sát.
4. Contract ở release sau khi không còn reader cũ.

10.8 Lab: đọc release như state machine

Lab thực hành

Không deploy. Mở `.github/workflows/deploy-production.yml` và `scripts/ops/remote-deploy.sh`, lập bảng cho từng bước: input, state mutation, checkpoint, failure cleanup và rollback. Tìm các điểm trước/sau migration và trước/sau switch symlink.

Checkpoint

Bạn phải mô tả được trạng thái host nếu npm install, backup, migration, restart hoặc readiness lần lượt thất bại. Nếu có trạng thái không rõ, automation chưa đủ an toàn.

10.9 Failure modes

- CI xanh nhưng artifact thiếu migration/static assets.
- Hai workflow production chạy đồng thời.
- Host key không được pin.
- Migration destructive làm app cũ không rollback được.
- Public health check qua CDN vẫn xanh trong khi origin sai release.

10.10 Tự kiểm tra

1. Vì sao build trên CI tốt hơn build lại trên host?
2. `systemctl restart` thành công khác readiness 200 thế nào?
3. Tại sao application rollback không tự động rollback database?

Hint đúc kết

- Release là chuỗi checkpoint có state rõ ràng.
- Deploy lock và concurrency control giải quyết hai lớp khác nhau.
- Migration tương thích ngược là nền tảng rollback an toàn.
- Secret không được đóng gói cùng artifact mà nó bảo vệ.

Tiếp theo: [Vận hành và xử lý sự cố](#).

11 Vận hành và xử lý sự cố

11.1 Mục tiêu

Giữ hệ thống có thể quan sát, phục hồi và thay đổi an toàn sau go-live. Operations không chỉ là restart; nó là vòng lặp detect → diagnose → mitigate → learn.

11.2 Bốn tín hiệu đầu tiên

- **Traffic:** request rate và pattern.
- **Errors:** HTTP 5xx, auth anomalies, job failure.
- **Latency:** p50/p95/p99; tách app và database khi có thể.
- **Saturation:** CPU, RAM, disk, DB connections, event loop/pool queue.

VPS nhỏ chưa cần monitoring stack lớn. Structured journal, health check, disk alert và external uptime check đã tạo giá trị. Thêm metrics stack khi câu hỏi vận hành không còn trả lời được từ những nguồn đó.

11.3 Evidence trước action

```
sudo systemctl status ai-learn-jlpt --no-pager
sudo journalctl -u ai-learn-jlpt --since '15 minutes ago' --no-pager
curl -i http://127.0.0.1:5050/health/live
curl -i http://127.0.0.1:5050/health/ready
df -h
free -h
```

Ghi timestamp, symptom, release SHA và action đã làm. Restart có thể giảm impact nhưng xoá evidence tạm thời; thu thập log/status trước nếu incident cho phép.

11.4 Playbook: app crash

1. Kiểm tra exit reason và restart count trong systemd/journal.
2. Xác định lỗi startup config, DB hay runtime exception.
3. Nếu liên quan release mới, xác nhận current và rollback artifact.
4. Nếu crash loop, dừng traffic/restart storm rồi điều tra.
5. Sau mitigate, thêm regression test hoặc guard.

11.5 Playbook: PostgreSQL không ready

1. /live và /ready phân biệt process với dependency.

2. `pg_isready`, `systemctl status postgresql`, journal và disk.
3. Kiểm tra connection count/long transaction, không tăng pool mù quáng.
4. Không restart DB nếu chưa biết recovery/checkpoint impact.
5. Nếu corruption/loss, phục hồi vào database mới từ backup verified.

11.6 Playbook: disk đầy

Disk đầy có thể làm app log lỗi, PostgreSQL ngừng write và backup fail. Xác định consumer bằng `du` với target cụ thể. Không xóa PostgreSQL data/WAL thủ công. Giảm log/backup cũ theo retention đã thiết kế, mở rộng disk nếu growth hợp lệ.

Cảnh báo

Không chạy lệnh xóa recursive trên `/`, `$HOME`, PostgreSQL data directory hoặc path ghép từ biến chưa validate. “Giải phóng disk nhanh” có thể biến outage thành mất dữ liệu.

11.7 Playbook: migration/release lỗi

- Migration transaction fail trước commit: app cũ vẫn active; sửa migration mới.
- Migration commit nhưng app mới fail: rollback app chỉ khi schema compatible.
- Release không ready: xem journal của đúng SHA, DB connectivity và assets.
- Không sửa migration đã apply; tạo migration correction.

11.8 TLS và certificate

Theo dõi expiry từ bên ngoài. `certbot renew --dry-run` kiểm tra renewal path. Nếu TLS lỗi, phân biệt DNS sai, port 80/443 bị chặn, Nginx config sai và rate limit CA. Không tắt HTTPS để “tạm chạy” auth production.

11.9 Backup operations

Backup hằng ngày giữ 14 ngày là baseline, không phải chiến lược hoàn chỉnh. Có ít nhất một copy ngoài VPS và định kỳ restore drill. Theo dõi tuổi file backup, size bất thường và exit status timer.

Recovery Point Objective (RPO) là lượng dữ liệu có thể mất; Recovery Time Objective (RTO) là thời gian phục hồi chấp nhận được. Tần suất backup và runbook phải xuất phát từ hai con số này.

11.10 Khi nào scale

- Tách PostgreSQL khi tài nguyên/backup/availability cần failure domain riêng.

- Thêm replica app khi CPU/availability cần, sau khi session/data đã shared.
- Thêm cache khi query plan/index không đủ và cache invalidation rõ.
- Thêm queue khi công việc dài/retry làm request không ổn định.

11.11 Lab: game day không phá dữ liệu

Lab thực hành

Trên môi trường test, lần lượt dừng app process, dùng DB URL sai và deploy một artifact cố ý thiếu config. Với mỗi tình huống ghi: alert/symptom, lệnh thu thập bằng chứng, automatic recovery, manual mitigation và regression guard. Không mô phỏng disk full hoặc kill PostgreSQL production.

11.12 Tự kiểm tra

1. Vì sao restart không phải root-cause analysis?
2. RPO/RTO ảnh hưởng backup thế nào?
3. Khi nào thêm replica app chưa giải quyết availability?

Hint đúc kết

- Quan sát theo traffic, errors, latency và saturation.
- Mọi incident nên tạo một guard mới: test, alert, limit hoặc runbook.
- Backup age là metric quan trọng hơn việc timer “active”.
- Scale sau khi đo bottleneck; thêm thành phần cũng thêm failure mode.

Tiếp theo: [Command cookbook](#).

12 Command cookbook

Chương này dùng để tra nhanh. Ký hiệu rủi ro: **R0** chỉ đọc, **R1** tạo output cục bộ, **R2** thay đổi service/dữ liệu có kiểm soát, **R3** có thể mất dữ liệu.

12.1 Git, dependency và build

Nơi	Lệnh	Mục đích	Risk/output
Local	<code>git status</code> <code>--short</code>	Xem thay đổi	R0
Local/CI	<code>npm ci</code>	Cài đúng lockfile	R1, tạo <code>node_modules</code>
Local/CI	<code>npm run check</code>	JS check + typecheck	R0/R1 cache
Local/CI	<code>npm test</code>	Unit + architecture	R0
Local/CI	<code>npm run build</code>	Tạo dist/	R1
Test	<code>npm run</code> <code>test:integration</code>	API smoke trên DB cô lập	R2, bắt buộc <code>TEST_DATABASE_URL</code>

`npm ci` phù hợp CI/release vì từ chối lockfile lệch; `npm install` phù hợp khi cố ý thay dependency và cập nhật lockfile.

12.2 PostgreSQL status và khám phá

```
pg_isready
psql "$DATABASE_URL" -XAtqc 'SELECT current_database(), current_user'
psql "$DATABASE_URL" -Xc '\dt'
psql "$DATABASE_URL" -Xc '\d users'
```

Nơi: local hoặc VPS. Risk R0. Output phải cho đúng database/user trước mọi thao tác tiếp theo.

```
psql "$DATABASE_URL" -XAtqc \
"SELECT count(*) FROM pg_stat_activity WHERE datname=current_database();"

```

Risk R0; dùng để điều tra connection pressure.

12.3 Database lifecycle

```
createdb ai_learn_jlpt_lab
DATABASE_URI=postgresql://localhost:5432/ai_learn_jlpt_lab npm run
↪ db:migrate
```

Nơi: local. Risk R2. Output migration phải là `APPLY` hoặc `SKIP` có chủ đích.

Cảnh báo

`dropdb` là R3. Chỉ chạy sau `preflight`:

```
target_db=ai_learn_jlpt_lab
case "$target_db" in
  *lab*|*test*)
    dropdb "$target_db"
    ;;
*)
  echo 'Refused'
  exit 1
  ;;
esac
```

12.4 Backup và restore

```
pg_dump -Fc --no-owner --no-acl "$DATABASE_URL" -f app.dump
pg_restore --list app.dump >/dev/null
shasum -a 256 app.dump > app.dump.sha256
```

Nơi: local/VPS. Risk R1; đọc DB và tạo file chứa dữ liệu nhạy cảm. Output dump phải quyền hạn chế và lưu ngoài Git.

```
createdb ai_learn_jlpt_restore_test
pg_restore --exit-on-error --no-owner --no-acl \
  -d postgresql://localhost:5432/ai_learn_jlpt_restore_test app.dump
```

Risk R2 trên DB mới. Không dùng `--clean` vào production nếu chưa có recovery plan được review.

12.5 HTTP và health

```
curl -i http://127.0.0.1:5050/health/live
curl -i http://127.0.0.1:5050/health/ready
curl --fail --silent --show-error https://learn.example.com/health/ready
```

Nơi: hai lệnh đầu trên VPS, lệnh cuối từ ngoài. Risk R0. So sánh internal và public giúp khoanh vùng Nginx/DNS/TLS.

12.6 systemd và journal

```
sudo systemctl status ai-learn-jlpt --no-pager
sudo journalctl -u ai-learn-jlpt -n 100 --no-pager
sudo journalctl -u ai-learn-jlpt -f
sudo systemctl list-timers 'ai-learn-jlpt*'
```

Risk R0. Restart là R2:

```
sudo systemctl restart ai-learn-jlpt
curl --fail http://127.0.0.1:5050/health/ready
```

Luôn theo restart bằng readiness; không coi command exit 0 là acceptance test.

12.7 Port, process và resource

```
ss -ltnp
lsof -iTCP:5050 -sTCP:LISTEN
ps -ef | rg 'dist/server.js'
df -h
free -h
```

ss/free phổ biến trên Linux; lsof dùng tốt trên macOS. Risk R0.

12.8 Nginx, DNS và TLS

```
sudo nginx -t
sudo systemctl reload nginx
dig +short learn.example.com
curl -Iv https://learn.example.com
sudo certbot renew --dry-run
```


nginx -t, DNS và curl là R0. Reload là R2 nhưng giữ connection tốt hơn restart. Chỉ reload sau config test thành công.

12.9 SSH và deploy

```
ssh-keygen -lf /etc/ssh/ssh_host_ed25519_key.pub
readlink -f /opt/ai-learn-jlpt/current
ls -lt /opt/ai-learn-jlpt/releases
```

Risk R0. Dùng fingerprint qua trusted console để tạo known-host secret.

Rollback app là R2 và phải xác nhận compatibility:

```
previous=/opt/ai-learn-jlpt/releases/REVIEWED_COMMIT_SHA
test -f "$previous/dist/server.js"
sudo -u deploy ln -sfn "$previous" /opt/ai-learn-jlpt/current.rollback
sudo -u deploy mv -Tf /opt/ai-learn-jlpt/current.rollback
↪ /opt/ai-learn-jlpt/current
sudo systemctl restart ai-learn-jlpt
curl --fail http://127.0.0.1:5050/health/ready
```

12.10 Cách dùng cookbook khi incident

1. Bắt đầu R0: status, log, health, disk, DB connection.
2. Ghi evidence và release SHA.
3. Chọn action R2 nhỏ nhất có khả năng giảm impact.
4. R3 chỉ sau backup, target validation và peer review.

Hint đúc kết

- Mọi lệnh automation cần set -euo pipefail và target cụ thể.
- Luôn phân biệt nơi chạy: local, CI, deploy user hay root.
- Lệnh thành công chưa phải hệ thống thành công; theo bằng checkpoint domain.
- Không copy lệnh R3 từ runbook mà bỏ preflight.

Tiếp theo: [Capstone và checklist](#).

13 Capstone và checklist

13.1 Mục tiêu

Ghép các phần thành một vòng đời duy nhất và tự đánh giá khả năng giải thích, thực hiện, chẩn đoán và phục hồi hệ thống.

13.2 Capstone: từ login tới recovery

```
login -> cookie -> request context -> authorization -> create note  
-> PostgreSQL -> build -> release -> readiness -> backup/restore
```

Lab thực hành

Môi trường: local server và database có tên chứa `capstone_test`. Không dùng credential/dump production.

13.2.1 1. Chuẩn bị

```
createdb ai_learn_jlpt_capstone_test  
export TEST_DATABASE_URL=  
↪ postgresql://localhost:5432/ai_learn_jlpt_capstone_test  
DATABASE_URL="$TEST_DATABASE_URL" npm run db:migrate
```

13.2.2 2. Identity và ownership

Chạy app bằng test DB. Đăng ký A/B với hai cookie jar. A tạo note; B thử truy cập ID đó; A đọc lại. Ghi status và query chứng minh owner không đổi.

13.2.3 3. Artifact

```
npm run check  
npm test  
npm run build  
DATABASE_URL="$TEST_DATABASE_URL" SMOKE_ENTRY=dist/server.js \  
node scripts/qa/smoke-test.js
```

Xác nhận `dist/server.js`, public HTML và migration runner tồn tại.

13.2.4 4. Backup/recovery

```
pg_dump -Fc --no-owner --no-acl "$TEST_DATABASE_URL" -f
↪ /tmp/capstone.dump
pg_restore --list /tmp/capstone.dump >/dev/null
createdb ai_learn_jlpt_capstone_restore_test
pg_restore --exit-on-error --no-owner --no-acl \
  -d postgresql://localhost:5432/ai_learn_jlpt_capstone_restore_test
↪ \
  /tmp/capstone.dump
```

Chạy migration + smoke trên DB restore. So sánh marker data và migration count.

13.2.5 5. Cleanup

```
dropdb ai_learn_jlpt_capstone_restore_test
dropdb ai_learn_jlpt_capstone_test
rm -f /tmp/capstone.dump
unset TEST_DATABASE_URL
```

Checkpoint

Capstone đạt khi bạn có evidence cho auth cookie, cross-user isolation, artifact, health, backup validity và restored application — không chỉ một danh sách command exit 0.

13.3 Maturity matrix

Mức	Đặc điểm	Bước tiếp theo khi có signal
Local VPS production	Một process, local DB Nginx, TLS, systemd, backup, CI/CD	Test DB, reproducible build External backup/monitoring
Resilient single region	DB managed/tách host, nhiều app replica	Pool/capacity/session review
Distributed	Queue/cache/object storage/observability	Chỉ thêm theo workload

Không nhảy level vì xu hướng. Mỗi level làm tăng chi phí deploy, data consistency, security và incident coordination.

13.4 Checklist kiến thức

- ☐ Vẽ được request flow từ browser tới DB.
- ☐ Phân biệt authentication, authorization, role và ownership.
- ☐ Giải thích password salt/hash và session token hash.
- ☐ Giải thích 401, 403 và non-owner behavior.
- ☐ Viết private query luôn scope `user_id`.
- ☐ Dùng transaction trên cùng DB client.
- ☐ Đọc constraint, index và query plan cơ bản.
- ☐ Tính tổng pool connection khi tăng replica.
- ☐ Viết migration forward và review lock/compatibility.
- ☐ Backup, validate và restore vào database mới.
- ☐ Phân biệt source, artifact và active release.
- ☐ Phân biệt liveness, readiness và process status.
- ☐ Mô tả rollback app và giới hạn với database.
- ☐ Thu thập evidence trước action trong incident.

13.5 Security review

- ☐ Password/token/credential không xuất hiện trong log hoặc response.
- ☐ Production cookie có `HttpOnly`, `Secure`, `SameSite` phù hợp.
- ☐ Login có rate limit/backoff theo threat model.
- ☐ Reset/verification token được hash, có TTL và one-time consume.
- ☐ Mọi private CRUD được test owner/non-owner.
- ☐ Admin action có access guard và audit.
- ☐ DB/app không public port không cần thiết.
- ☐ Deploy key và sudo theo least privilege.
- ☐ Secret không nằm trong source/artifact.

13.6 Database và go-live review

- ☐ Constraint phản ánh invariant quan trọng.
- ☐ Query dùng parameter binding.
- ☐ Index gắn với query thật và đã xem plan.
- ☐ Migration đã thử trên bản sao dữ liệu.
- ☐ Predeploy backup verified.
- ☐ Restore drill thành công.
- ☐ DNS, TLS, Nginx và external readiness đều đúng.
- ☐ systemd service, health timer và backup timer active.
- ☐ Login, CMS, private data và image path được smoke test.
- ☐ Có bản backup ngoài VPS và owner nhận alert.

13.7 Câu hỏi tình huống

1. `/live` 200 nhưng `/ready` 500: thu thập gì trước?

2. Release mới migration thành công nhưng app fail: điều kiện nào cho phép rollback?
3. User B update được note A nhưng không list được: boundary nào có khả năng sai?
4. Tăng từ 1 lên 8 replica và DB bắt đầu từ chối connection: tính lại gì?
5. Backup timer active nhưng file mới nhất ba ngày tuổi: hệ thống đang xanh không?

13.8 Những thứ chưa cần vội dùng

- **JWT:** session DB đang đáp ứng browser app và revoke tốt.
- **Redis:** chưa có query/latency evidence hoặc cache policy.
- **ORM:** raw SQL nhỏ, rõ và được test.
- **Microservices:** module boundary chưa cần network boundary.
- **Kubernetes:** một VPS/systemd còn đáp ứng topology.
- **MFA:** ưu tiên rate limit, reset lifecycle và admin threat trước; thêm khi risk yêu cầu.

Hint đúc kết

- Nắm invariant và failure mode trước syntax công cụ.
- Một hệ thống trưởng thành khi có thể thay đổi và phục hồi có kiểm chứng.
- “Chưa cần” là kết luận kỹ thuật, không phải thiếu tham vọng.
- Khi gặp codebase mới: vẽ request flow, data ownership, failure boundary và operational loop — bốn bản đồ này mở gần như mọi ngõ ngách.

Lời kết

Một hệ thống đáng tin cậy không được tạo nên bởi số lượng framework, mà bởi các boundary rõ ràng, dữ liệu có chủ sở hữu, thay đổi có thể kiểm chứng và quy trình phục hồi đã được diễn tập.